🧩 E20 — Recursividad y Memoization

🎯 **Obejtivos:**

- Comprender cómo funciona la recursividad mediante el cálculo de la serie de Fibonacci.
- Identificar el problema del crecimiento exponencial de llamadas en la recursión directa.
- Implementar **memoization** para optimizar la recursión.

📘 Teoría y conceptos

1. Recursividad

Una función es recursiva cuando **se llama a sí misma** para resolver un problema más pequeño del mismo tipo.
La serie de Fibonacci se define como:

$$F(0) = 0,\ F(1) = 1,\ F(2) = 2,\ F(n) = F(n-1) + F(n-2)$$

La versión recursiva directa es elegante, pero **muy ineficiente**, porque recalcula los mismos valores muchas veces.

2. Problema del crecimiento exponencial

La recursión directa tiene un número de llamadas que crece aproximadamente como (2^n).
Por eso funciona para valores pequeños (ej. 10, 20), pero se vuelve extremadamente lenta para valores mayores (ej. 35, 40).

3. Memoization

Memoization consiste en **guardar los resultados ya calculados** para no repetir trabajo.
Ventajas:

- Reduce el tiempo de ejecución de exponencial a lineal.
- Reduce drásticamente el número de llamadas recursivas.

📝 Instrucciones

Parte 1 — Fibonacci recursivo directo

1. Implementa la función recursiva sin optimización.
2. Agrega un contador global para medir cuántas veces se llama la función.
3. Mide el tiempo de ejecución con time.time().
4. Prueba con un valor pequeño (ej. 10).
5. Luego prueba con un valor grande (ej. 35 o 40) y observa el tiempo.

Parte 2 — Fibonacci con memoization

1. Implementa una versión recursiva que use un diccionario para guardar resultados.
2. Agrega un contador de llamadas.
3. Mide el tiempo de ejecución.
4. Compara con la versión anterior.
5. Reflexiona sobre la diferencia en rendimiento.

Código 1 — Fibonacci recursivo directo

```
# Fibonacci recursivo directo
import time

def fib_recursivo(n):
    global llamadas
    llamadas += 1
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fib_recursivo(n - 1) + fib_recursivo(n - 2)



n = int(input("Ingresa un número para Fibonacci (recursivo directo): "))
llamadas = 0
inicio = time.time()
resultado = fib_recursivo(n)
fin = time.time()

print("\nRESULTADOS (Recursivo directo)")
print("Fibonacci de", n, "=", resultado)
print("Llamadas realizadas:", llamadas)
print("Tiempo de ejecución:", fin - inicio, "segundos")
```

Código 2 — Fibonacci con memoization

```
# Fibonacci recursivo con memoization
import time

def fib_memo(n, memo):
    global llamadas_memo
    llamadas_memo += 1
    if n in memo: # Si ya está guardado, lo regresamos
        return memo[n]
    if n == 0:
        memo[0] = 0
        return 0
    if n == 1:
        memo[1] = 1
        return 1
    # Guardamos el resultado para no recalcularlo
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]


n = int(input("Ingresa un número para Fibonacci (memoization): "))
llamadas_memo = 0
memo = {}
inicio = time.time()
resultado = fib_memo(n, memo)
fin = time.time()
print("\nRESULTADOS (Memoization)")
print("Fibonacci de", n, "=", resultado)
print("Llamadas realizadas:", llamadas_memo)
print("Tiempo de ejecución:", fin - inicio, "segundos")
```